Grocery Delivery Management System

System Requirements Documentation

Susan Rogers

SP26-NW-INFO-C450-24493

Jafrina Jabin

03/09/2026

Table of Contents

Grocery Delivery Management System

Susan Rogers

Problem Statement

Grocery shopping is necessary but time-consuming for many people, including busy professionals, elderly individuals, families, and customers with limited mobility. Traditional shopping requires travel, time, and physical effort. Many local grocery stores do not have an affordable way to offer online ordering and delivery. This leads to inaccurate stock information, delayed deliveries, and poor communication with customers.

Objectives of the System

Provide an easy-to-use grocery delivery platform for customers and local grocery stores. Reduce manual errors, improve inventory accuracy, streamline order fulfillment, and increase customer satisfaction.

System Functionality Overview

Customers browse and search products, add items to a cart, place delivery orders, and track order status. Managers maintain product listings and inventory, review orders, and assign deliveries. Delivery staff update delivery progress so customers can see real-time status.

System Requirements

• Register and securely log in
• Browse products by category and search by keywords
• Add items to a cart and place delivery orders
• View order history and delivery status
• Receive notifications for confirmations and updates
• Managers add, update, or remove products
• Inventory automatically updates when orders are placed
• Assign delivery orders and update delivery status

Typical Customers

Customers ordering groceries for home delivery, grocery store managers, and delivery personnel.

Project Planning (Software, Hardware, Network)

Software: Java application developed in NetBeans using in-memory data storage

Development Approach

The system was developed in Java using Swing for the interface and Java ArrayLists in the DataStore class for in-memory data management

Project Plan

1. Weeks 1–2: finalize requirements,design system data model
2. Weeks 3–4: implement user registration and login
3. Weeks 5–7: implement product management, inventory, and order placement
4. Week 8: testing and midterm demo
5. Weeks 9–11: implement delivery assignment and tracking
6. Weeks 12–14: testing, bug fixes, and improvements
7. Week 15: final testing and presentation

Estimated Cost and Business Value

The primary costs involve development time and basic hosting services. The system provides strong business value by enabling local grocery stores to operate online, improving efficiency, reducing errors, and increasing customer satisfaction.

Grocery Delivery Management System

System Design Overview and Communication Document

Susan Rogers

1. Customer Problem Statement

Purchasing groceries is an essential activity but sometimes requires a significant investment of time and effort. Customers such as working professionals such as myself, older adults, households with young children, and people who have disabilities most times, experience difficulty traveling to stores, navigating aisles, and waiting in checkout lines. Although online grocery services are available through large retailers, many small and local grocery stores don't have cost-effective systems to support online ordering and delivery.

From a customer's perspective, an effective grocery delivery system should make it possible to look for products, submit orders remotely, and receive dependable updates about delivery progress. Customers expect that the exact product availability is accurate and that deliveries occur within a reasonable timeframe. Store managers, on the other hand, expect a system that makes work, minimizes inventory discrepancies, and improves coordination between ordering and delivery. Overall, the end goal of this system is to improve life overall for customers while helping local grocery stores operate more efficiently.

2. Glossary of Terms

Customer: An individual who selects grocery items and submits delivery orders

Manager: A store representative responsible for overseeing products, inventory, and orders

Delivery Staff: Employees tasked with delivering grocery orders to customers

Product: A grocery item offered for purchase within the system

Inventory: The available quantity of products maintained by the store

Order: A completed request for grocery items to be delivered

Order Status: An indicator representing the current stage of an order

Notification: A system-generated message informing customers of order updates

3. System Requirements

3.1 Functional Requirements

- Users shall be able to create accounts and authenticate securely.

- Customers shall be able to view products organized by category.

- Customers shall be able to locate products using a search function.

- Customers shall be able to select items and store them in a shopping cart.

- Customers shall be able to submit orders for grocery delivery.

- Customers shall be able to review previously placed orders.

- Customers shall be able to monitor the progress of active deliveries.

- Customers shall receive system notifications related to their orders.

- Managers shall be able to create, update, and remove product listings.

- The system shall automatically adjust inventory when orders are placed.

- Managers shall be able to view and manage incoming orders.

- Managers shall be able to assign delivery tasks to delivery staff.

- Delivery staff shall be able to update the status of assigned deliveries.

3.2 Nonfunctional Requirements (FURPS)

- Functionality: The system shall support the accuracy of orders and inventory.

- Usability: The system should be easy to use and get to for users with varying technical skills.

- Reliability: The system shall consistently maintain correct and up-to-date information.

- Performance: System responses shall occur within a small amount of time limits.

- Supportability: The system should be structured to allow flexibility for upgrades.

4. Plan of Work

Project development followed the planned schedule established during the proposal phase. Initial efforts focused on preparing the development environment and refining system requirements. Subsequent planning addressed core features including user access, product management, inventory handling, and order processing. Additional work is planned for delivery coordination, system testing, and final refinements before project completion.

5. When to Start Coding

Implementation starts after completing requirements analysis and system design activities. Development initially concentrated on the server-side application logic, followed by user-facing components. Data storage integration and testing were conducted alongside development to ensure the system runs smoothly and that data remains consistent within the in-memory DataStore.

Use Case Diagram

The following use case diagram illustrates how different actors interact with the Grocery Delivery Management System. It shows primary system functionalities and shows how customers, store managers, and delivery staff interact with the system through required and optional behaviors.

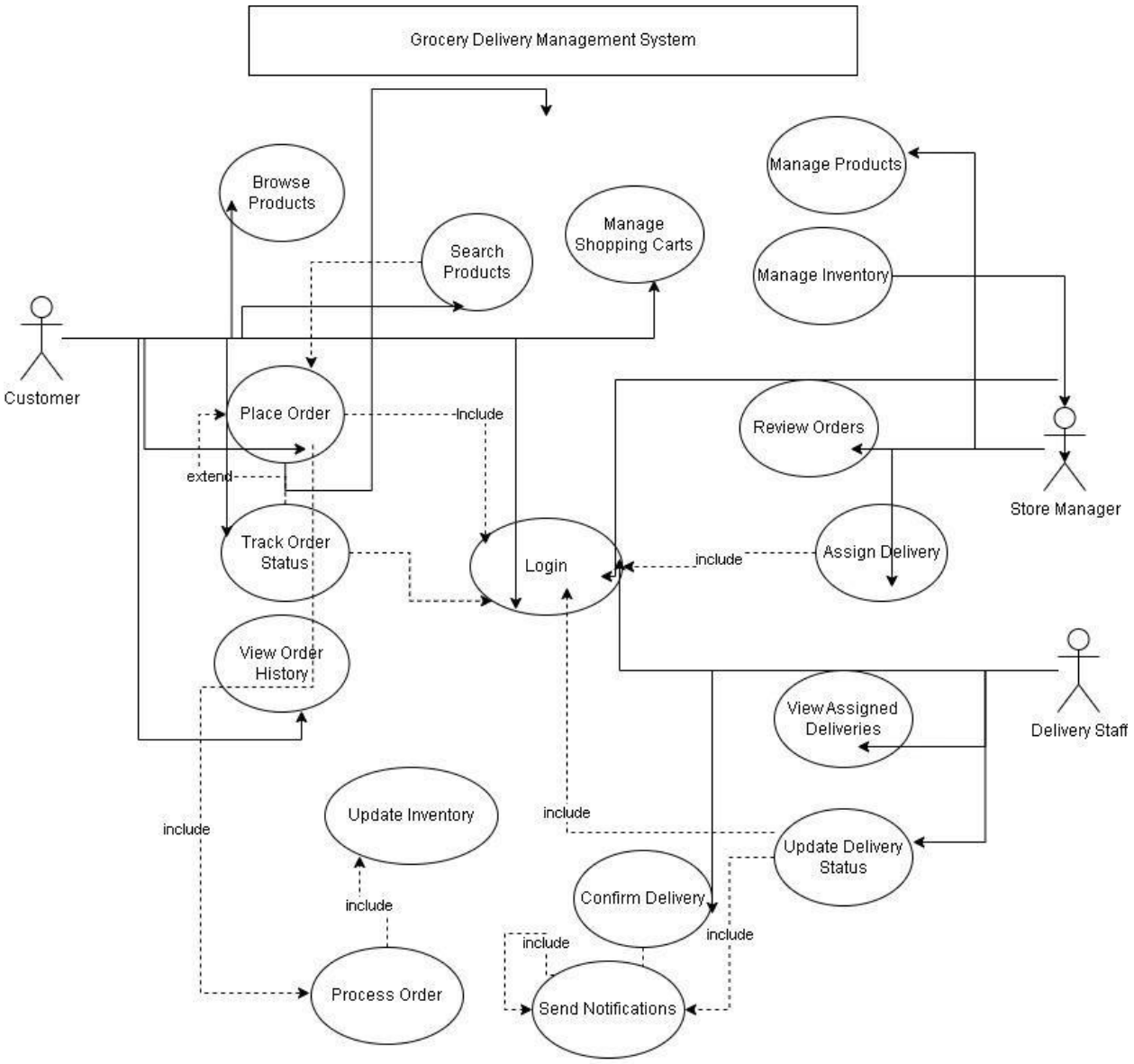


Figure: Grocery Delivery Management System Use Case Diagram

Class Diagram

The following class diagram shows the main classes used in the Grocery Delivery Management System and how they relate to one another. It highlights users, orders, products, and delivery components.

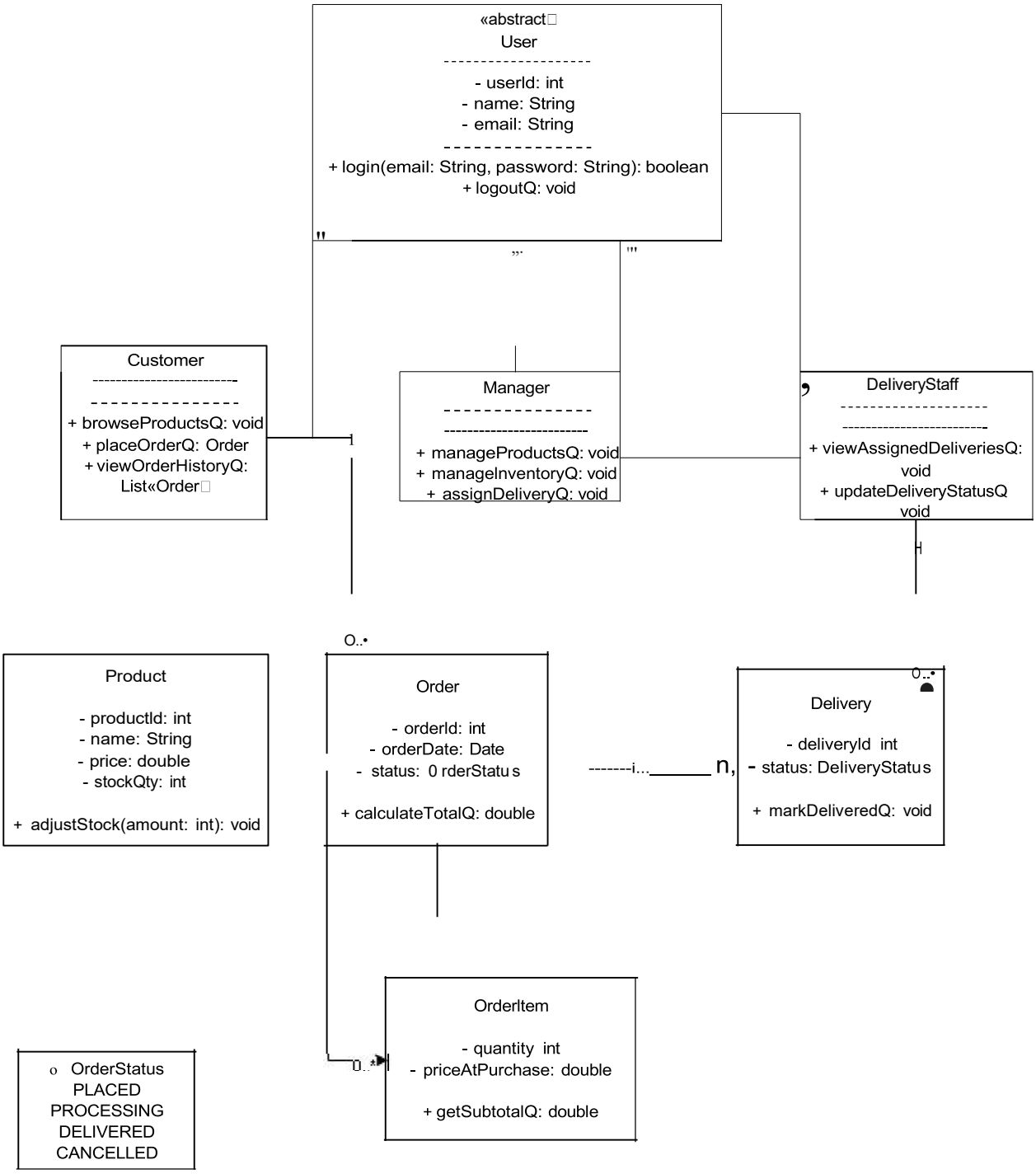


Figure: Grocery Delivery Management System Class Diagram

Grocery Delivery Management System

Module 6 – System Sequence Diagrams and Activity Diagrams

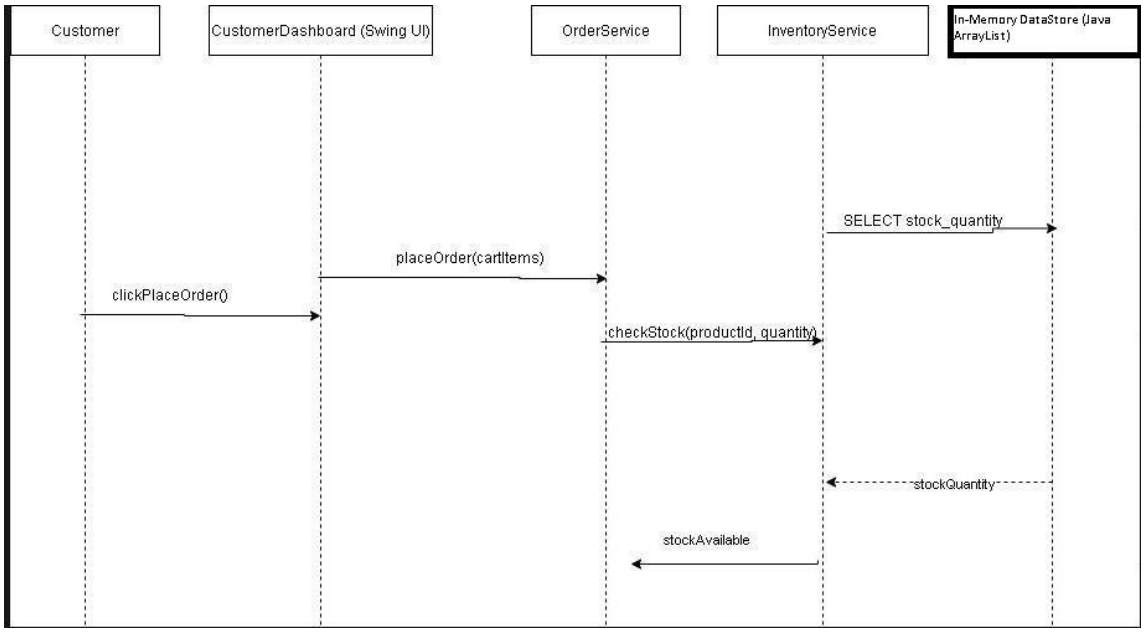
Susan Rogers

Introduction

This document communicates the design solutions and implementation approaches for the Grocery Delivery Management System. It presents UML system sequence diagrams and UML activity diagrams for two main and complex use cases, in accordance with the assignment requirements.

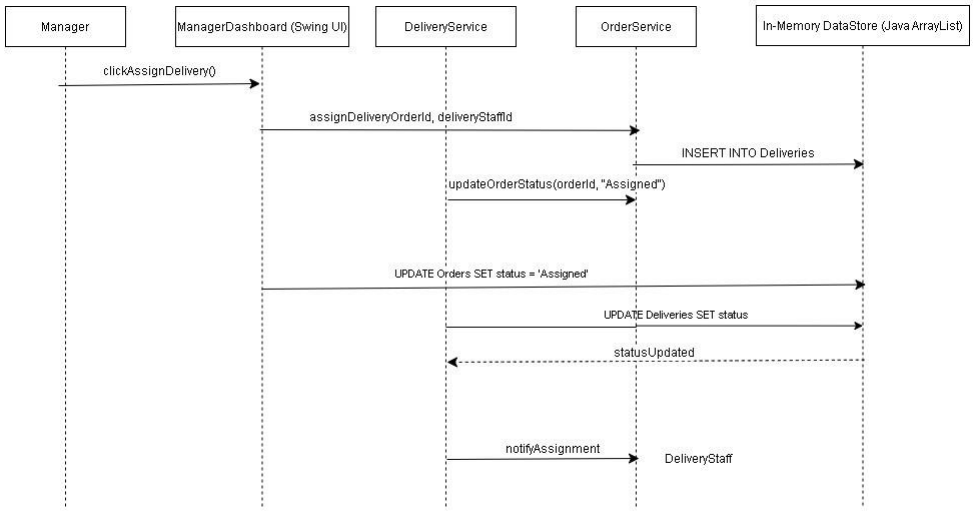
System Sequence Diagram – Use Case 1: Place Grocery Delivery Order

This sequence diagram illustrates how a customer places a grocery delivery order. It shows interactions between the Customer, CustomerDashboard (Swing UI), OrderService, InventoryService, and In-memory DataStore (Java ArrayList).



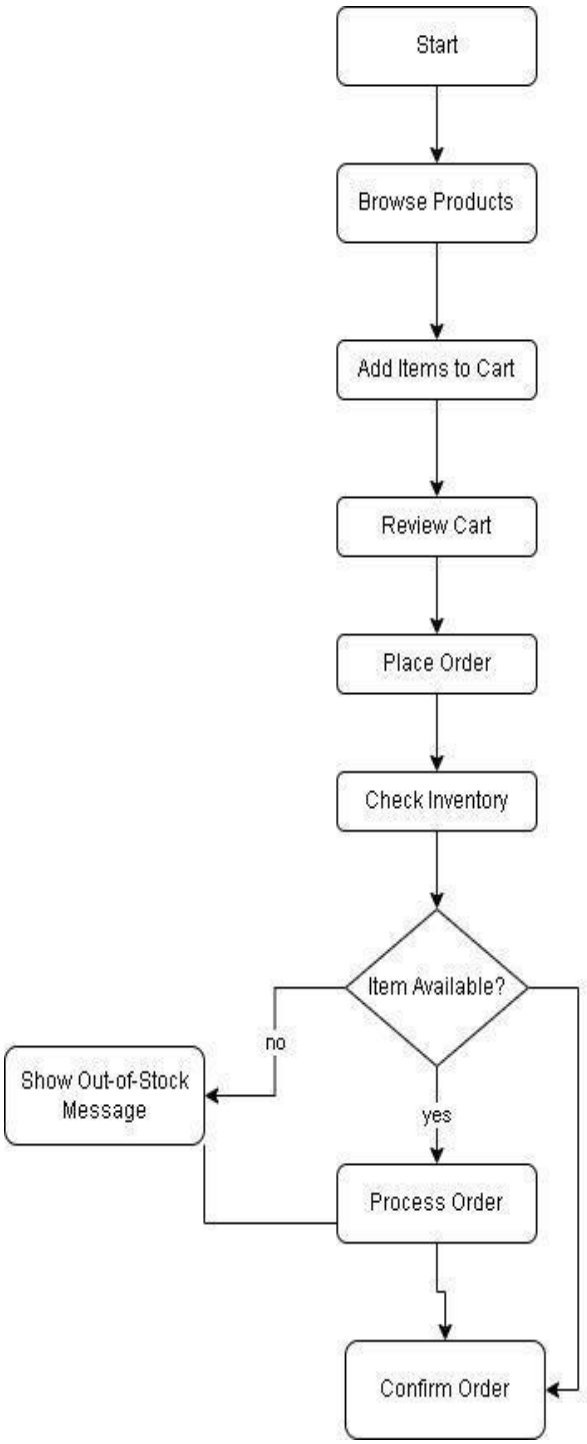
System Sequence Diagram – Use Case 2: Assign Delivery / Track Delivery

This sequence diagram illustrates how a manager assigns delivery staff to an order and how delivery status updates are handled within the system.



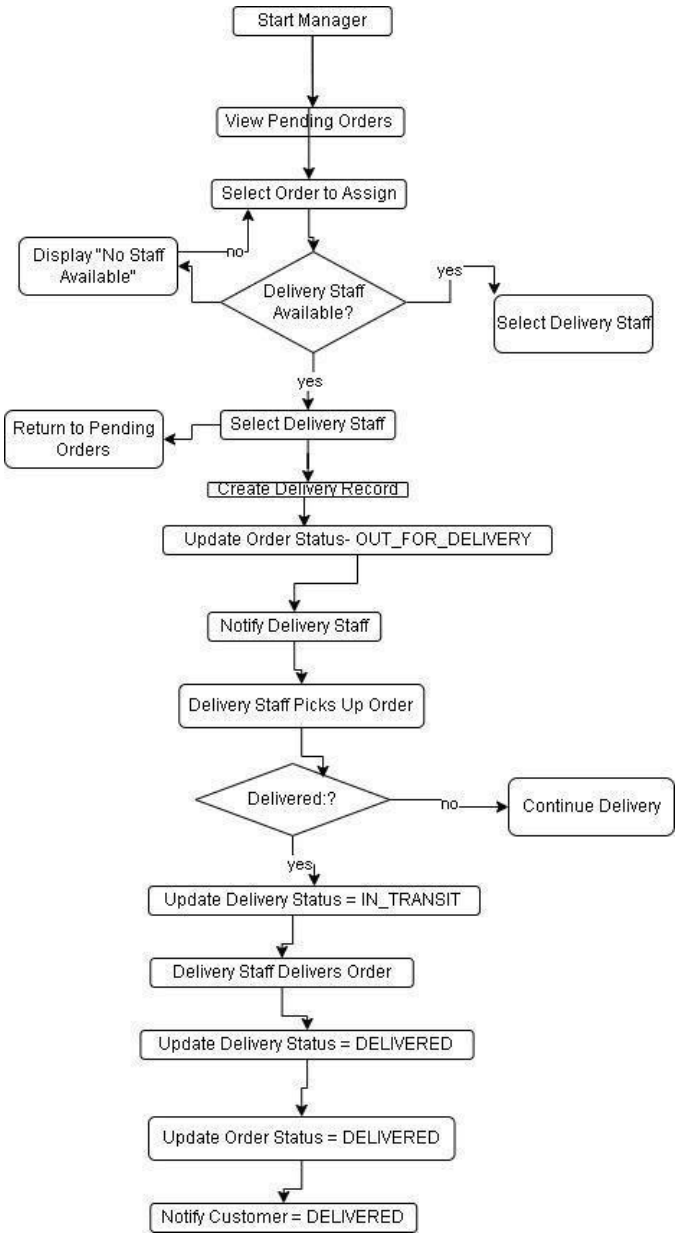
Activity Diagram – Use Case 1: Place Grocery Delivery Order

This UML activity diagram represents the workflow for placing a grocery delivery order, including browsing products, adding items to the cart, inventory validation, order processing, and order confirmation.



Activity Diagram – Use Case 2: Assign Delivery / Track Delivery

This UML activity diagram shows the workflow for assigning delivery staff, tracking delivery progress, updating delivery and order statuses, and notifying relevant parties.



Conclusion

This document provides a clear representation of the system design and workflow for the Grocery Delivery Management System. The included UML sequence and activity diagrams demonstrate how the system supports core grocery ordering and delivery processes.

Grocery Delivery Management System

User Interface Specification

Susan Rogers

Purpose of This Document

This document describes how people use the Grocery Delivery Management System from a user interface perspective. It explains what users see on the screen and how they move through the system while completing everyday tasks. No programming code or technical implementation details are included.
The interface is designed to be straightforward and easy to navigate. Customers, managers, and delivery staff should be able to complete their work with minimal effort and without unnecessary confusion. The interface designs described here are based on the system requirements established earlier in the project.

2. Selected Use Cases for User Interface Design

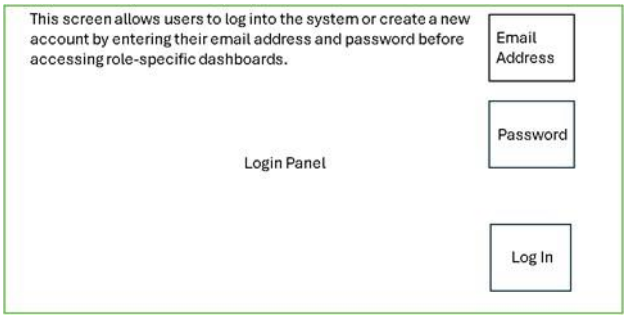
We selected these five use cases because they cover the most common and important ways users interact with the system:

- Register / Log In
- Browse and Search Products
- Place Grocery Delivery Order
- Assign Delivery Order
- Track Delivery Status

Together, these use cases represent the core functions of the Grocery Delivery Management System and reflect the workflows discussed in earlier design materials.

3. Preliminary User Interface Design

3.1 Use Case 1: Register / Log In



Primary Actors: Customer, Manager, Delivery Staff
Interface Description:
The login screen appears when a user first accesses the system. Before using any system features, users are required to enter valid login information. This screen provides the following options:

- Email address input field
- Password input field
- “Log In” button
- “Create Account” link

New users go to a registration screen where they enter the following information:

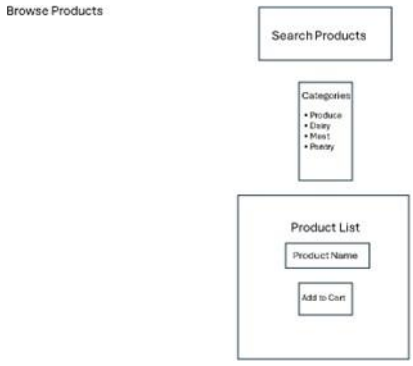
- Name
- Email address
- Password
- Password confirmation
- User role (Customer, Manager, or Delivery Staff)

Navigation path:
Login Screen → User Dashboard (based on assigned role)
System feedback:

- A message is displayed when a user enters incorrect login information
- A confirmation message is shown once the user successfully signs in

---

3.2 Use Case 2: Browse and Search Products



Primary Actor: Customer
Interface Description:
The product browsing screen lets customers view grocery items that are available for ordering. Items are organized by category to help users find products more easily. Each product listing includes:

- Product name
- Price
- Available quantity
- An "Add to Cart" button

A search bar at the top lets customers find items by typing keywords.
Navigation path:
Customer Dashboard → Browse Products → Product List
System Feedback: Search results update as the customer enters search terms

- An "Out of Stock" message is shown when an item cannot be ordered

3.3 Use Case 3: Place Grocery Delivery Order

Primary Actor: Customer

Interface description:
The shopping cart screen displays all items the customer has selected. This screen includes:

- A list of selected grocery items
- Quantity controls for each item
- Individual item subtotals
- The overall order total
- A “Place Order” button

After submitting the order, the confirmation screen shows:

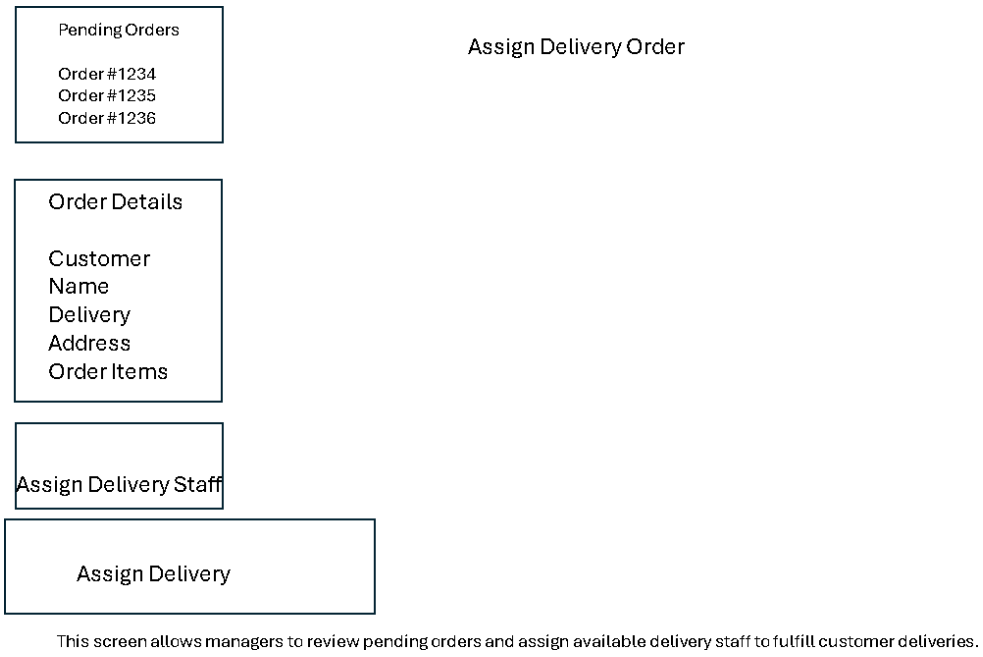
- A unique order number
- Current delivery status
- Estimated delivery time

Navigation Path:
Browse Navigation path: Cart → Place Order → Order Confirmation
System feedback:

- A message confirming that inventory is available
- A notification indicating that the order was successfully placed

This interface follows the order processing flow shown in the system’s activity and sequence diagrams.

3.4 Use Case 4: Assign Delivery Order



Primary Actor: Manager

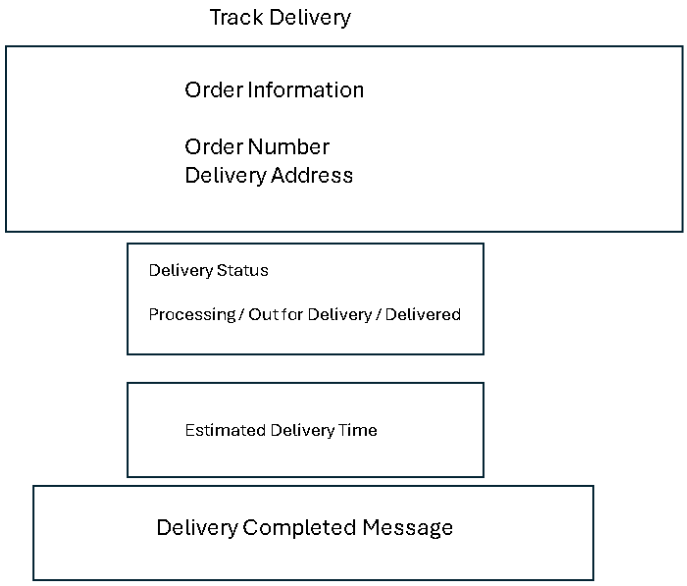
Interface Description: The dashboard displays all orders waiting to be assigned. When an order is selected, a delivery assignment screen appears showing:

- Order details
- A list of available delivery staff
- An “Assign Delivery” button

Navigation path:
Manager Dashboard → Pending Orders → Assign Delivery
System feedback:

- A confirmation message after a delivery assignment is completed
- Automatic update of the order status

3.5 Use Case 5: Track Delivery Status



This screen allows customers to monitor the current status of their grocery delivery and view estimated delivery information.

Primary Actor: Customer
Interface description:
The delivery tracking screen allows customers to check the progress of their orders. The following information is displayed:

- Current delivery status (Processing, Out for Delivery, Delivered)
- Delivery progress updates
- A delivery completion message

Navigation path:
Customer Dashboard → Order History → Track Delivery
System feedback:

- Notifications when the delivery status changes
- A final confirmation when delivery is completed

4. User Effort Estimation

The scenarios below estimate how much effort users need to complete common tasks. Effort is measured by counting mouse clicks and keystrokes for each scenario.

Scenario 1: Customer Places a Grocery Order

Action	Clicks	Keystrokes
Log in	1	15
Search Products	1	8
Add item to cart	1	0
View Cart	1	0
Place order	1	0
Total	5 clicks	23 keystrokes

Scenario 2: Manager Assigns Delivery Order

Action	Clicks	Keystrokes
Log in	1	15
View pending orders	1	0
Select Orders	1	0
Assign delivery staff	1	0
Total	4 Clicks	15 keystrokes

Scenario 3: Customer Tracks Delivery

Action	Clicks	Keystrokes
Log in	1	15
Open order history	1	0
Select Orders	1	0
View status	0	0
Total	3 Clicks	15 keystrokes

5. Conclusion

This User Interface Specification explains how the Grocery Delivery Management System supports user interaction through clear layouts and simple navigation paths. The interface

designs were created to support different user roles while keeping the number of needed actions as low as possible.
The designs here present a practical starting point for future interface development and show how usability was considered throughout the system design.

Grocery Delivery Management System

Traceability Matrix

Purpose

The traceability matrix's goal is to illustrate how each system requirement is addressed within the defined use cases. Its purpose is to ensure that every requirement is accounted for and that every use case is linked to a specific system need. Following this way helps maintain a clear and organized system design which avoids unnecessary or unsupported functionality. Additionally, the matrix highlights which use cases are linked to higher-priority requirements. These priority values assist in planning the order of feature development and in determining which use cases should be prioritized during the final system demonstration.

System Requirements

Req No.	Priority Weight (1–5)	Description
REQ1	5	Users shall be able to register and log in securely
REQ2	5	Customers shall be able to browse grocery products
REQ3	4	Customers shall be able to search for products
REQ4	5	Customers shall be able to add items to a shopping cart
REQ5	5	Customers shall be able to place grocery delivery orders
REQ6	4	Customers shall be able to view order history
REQ7	4	Customers shall be able to track delivery status
REQ8	5	Managers shall be able to add, update, or remove products
REQ9	4	Inventory shall automatically update when orders are placed
REQ10	4	Managers shall be able to assign delivery orders
REQ11	3	Delivery staff shall be able to update delivery status
REQ12	3	Customers shall receive system notifications

Use Cases

UC No.	Description
UC1	Register / Log In
UC2	Browse Products

UC No.	Description
UC3	Search Products
UC4	Add Items to Cart
UC5	Place Grocery Delivery Order
UC6	View Order History
UC7	Track Delivery Status
UC8	Manage Products
UC9	Update Inventory
UC10	Assign Delivery Order
UC11	Update Delivery Status
UC12	Receive Notifications
UC13	Log Out

Traceability Matrix

Req \ UC	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8	UC9	UC10	UC11	UC12	UC13
REQ1 (5)	X												X
REQ2 (5)		X											
REQ3 (4)			X										
REQ4 (5)				X									
REQ5 (5)					X				X			X	
REQ6 (4)						X							
REQ7 (4)							X					X	
REQ8 (5)								X					
REQ9 (4)					X				X				
REQ10 (4)										X			
REQ11 (3)											X		
REQ12 (3)					X		X					X	

Priority Weight Summary (Same Logic as Sample)

Use Case	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8	UC9	UC10	UC11	UC12	UC13
Max PW	5	5	4	5	5	4	4	5	4	4	3	3	5
Total PW	5	5	4	5	17	4	11	5	8	4	3	6	5

Implementation Priority Discussion

In relation to the traceability matrix, Use Case 5: Place Grocery Delivery Order is the biggest total priority weight. This use case supports several high-priority requirements, including order placement, inventory updates, delivery tracking, and notifications. Because of its central role in the system, this use case should be implemented and demonstrated first.

Other high-priority use cases include Register / Log In, Browse Products, Manage Products, and Track Delivery Status, as they support essential system functionality for customers and managers. Lower-priority use cases, such as updating delivery status and notifications, are still necessary but can be scheduled after the core workflows are complete.

System Architecture and System Design

1. Architectural Styles

The Grocery Delivery Management System follows a **layered architectural design** consisting of a presentation layer, an application logic layer, and a data management layer.

The system is implemented as a **Java desktop application using NetBeans and the Swing user interface framework**. This build separates user interaction from system processing and data management, which improves system organization and maintainability.

Presentation Layer (User Interface)

The presentation layer provides the graphical interface that users interact with. The interface is built using **Java Swing components**, including buttons, text fields, and forms. In using this interface, users can log in, browse grocery products, place orders, manage inventory, and track deliveries.

Application Logic Layer

The application logic layer contains the system services responsible for executing system operations. This layer processes user requests, validates information, manages orders, and coordinates delivery assignments.

Service components in this layer comprise modules responsible for order processing, inventory management, and delivery coordination.

Data Management Layer

The data management layer is responsible for maintaining system data during the time the application is running. The system stores information through a **DataStore class**, which uses **Java ArrayLists** to manage objects in memory. These lists temporarily contain data such as user accounts, product information, customer orders, and delivery assignments while the program is in operation.

Using this layered structure helps separate different responsibilities within the system. This organization also makes the system easier to maintain and allows future updates or improvements to be added more easily.

2. Identifying Subsystems

The Grocery Delivery Management System can be divided into several functional subsystems.

User Interface Subsystem

This subsystem manages user interaction with the system. It displays screens for login, product browsing, shopping cart management, order placement, and delivery tracking.

User Management Subsystem

This subsystem manages authentication and user roles within the system. It supports three primary user types:

- Customers
- Store Managers
- Delivery Staff

The subsystem makes certain that each user role has access to the appropriate system functions.

Product and Inventory Management Subsystem

This subsystem is responsible for handling product listings and tracking inventory levels. Store managers have the ability to add new products, update existing product information, or remove items when necessary. The system also adjusts inventory quantities automatically whenever a customer order is placed.

Order Processing Subsystem

This subsystem is responsible for managing customer orders. It processes the items selected by the customer, determines the total cost of the order, checks that the requested products are available, and saves the completed order information in the system.

Delivery Management Subsystem

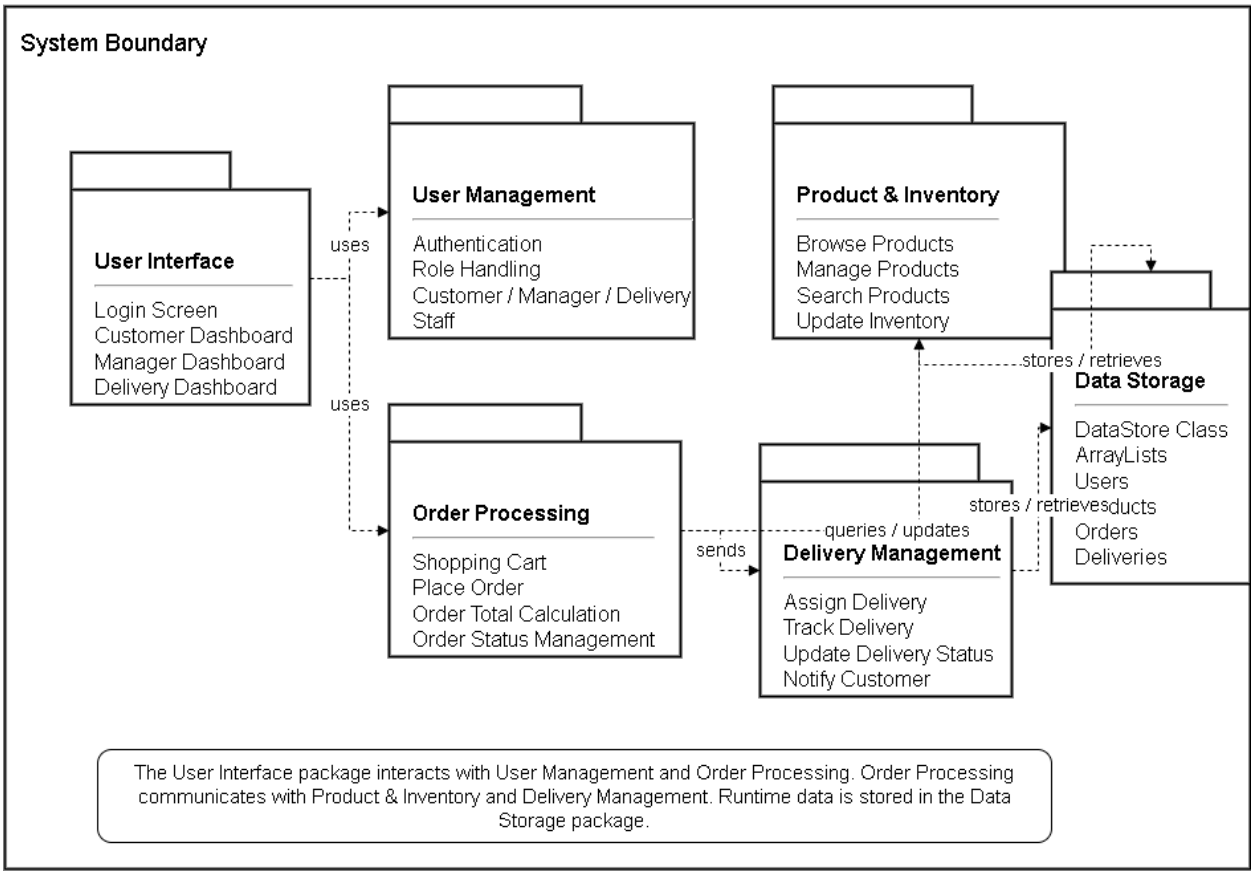
This subsystem coordinates delivery activities. Store managers assign deliveries to delivery staff members, and delivery staff update the delivery status as orders move through the delivery process.

Data Storage Subsystem

This subsystem manages system data during program execution. The **DataStore class stores objects such as users, products, orders, and deliveries using Java collections.**

Here is the UML package diagram for this system, which shows the user interface subsystem interacting with the application service subsystems, which then interact with the data storage subsystem.

UML Package Diagram
Grocery Delivery Management System



3. Mapping Subsystems to Hardware

The Grocery Delivery Management System operates on **a single computer system**.

All subsystems—including the user interface, application logic, and data storage—run within the same local machine. Because the system is designed as a desktop application for demonstration and academic purposes, it does not require distributed deployment across multiple machines.

4. Persistent Data Storage

The Grocery Delivery Management System requires data storage during the execution of the application.

Important system objects stored during runtime include:

- Users
- Products
- Orders
- Order Items
- Delivery Assignments
- Inventory information

The system stores these objects using **Java ArrayLists within the DataStore class**. These collections preserve system data while the application is running.

When the program stops running, the stored data is cleared. This approach eases development and testing for the purposes of the project.

5. Network Protocol

The Grocery Delivery Management System runs entirely on **one computer**.

Because the application runs locally and does not communicate with other computers or external servers, a network communication protocol is not required.

---

6. Global Control Flow

Execution Orders

The system follows an **event-driven design**.

Users interact with the application through graphical interface components such as buttons, menus, and form fields. When a user performs an action, the system responds by triggering the corresponding operation.

Examples of these interactions include:

- Selecting the **Log In** button to start the authentication process.
- Choosing a product and clicking **Add to Cart** to place the item in the shopping cart.

- Clicking **Place Order** to submit the order and update the inventory records.
- Managers selecting **Assign Delivery** triggers delivery assignment processing.

Because the system is event-driven, users may perform actions in different sequences depending on their needs.

Time Dependency

The system does not rely on timers or scheduled processes.

Instead, system actions is triggered by user actions. When a user performs an action, the system processes the request immediately and updates the interface accordingly.

This means the system functions as an **event-response system rather than a real-time system**.

This means the system reacts to user actions as they occur instead of operating as a real-time system with strict timing requirements.

Concurrency

The current version of the system does not use multiple threads.
All program operations are carried out in sequence within the main application thread.
Since the application runs locally and is designed to support a single active user session, this approach is sufficient for the needs of the system.

---

7. Hardware Requirements

The Grocery Delivery Management System relies on basic computing resources in order to operate correctly.

Display Requirements

The system requires a color monitor so that the graphical interface can be displayed clearly.

Minimum resolution: **1024 × 768 pixels**

Computer System

The application can run on a standard **desktop or laptop computer** capable of running Java-based software.

Memory Requirements

At minimum **2 GB of RAM** needs to be available to allow the application and its interface to operate properly.

Storage Requirements

The system needs roughly **500 MB of available disk space** for the program files and supporting components.

Software Requirements

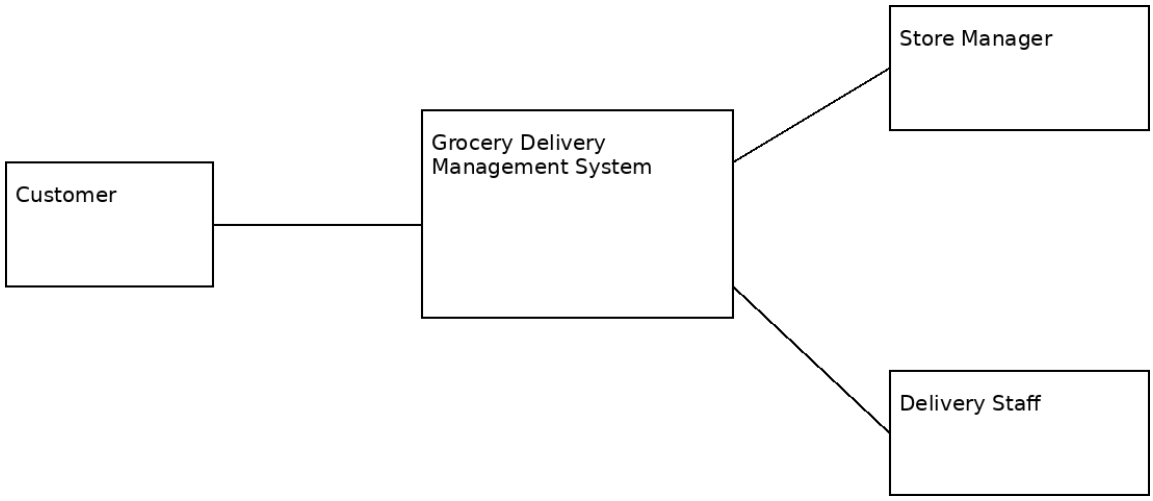
- Java Development Kit (JDK)
- NetBeans IDE
- Java Swing libraries

These components provide the tools required to run the application and support the program's interface and functionality.

System Design Specification
Part A: System Overview
System Description
The Grocery Delivery Management System is designed to support online grocery ordering and delivery operations for local stores. The system allows customers to browse products, place delivery orders, and track order status. Store managers are responsible for overseeing product listings, managing inventory, reviewing orders, and assigning deliveries. Delivery staff update delivery progress so that customers and managers can monitor order status.
The system provides a centralized solution that improves efficiency, cuts manual errors, and enhances communication between users.
The design follows a layered structure that separates the user interface, system processing, and data storage components. This approach assists keep the system organized and easier to maintain.

System Goals and Objectives
- Provide an easy-to-use grocery ordering system
- Allow customers to browse and purchase products
- Enable accurate inventory tracking
- Support order processing and delivery coordination
- Allow delivery tracking in real time
- Improve communication between customers, managers, and delivery staff
- Lower manual errors and improve efficiency

Part B: Design Integration
Context Diagram



The Grocery Delivery Management System interacts with three main outside entities:

Outside Entities

Customer

- Searches and browses products
- Places orders
- Tracks delivery status

Store Manager

- Updates product listings
- Manages inventory
- Reviews orders
- Assigns delivery staff

Delivery Staff

- Views assigned deliveries
- Updates delivery status

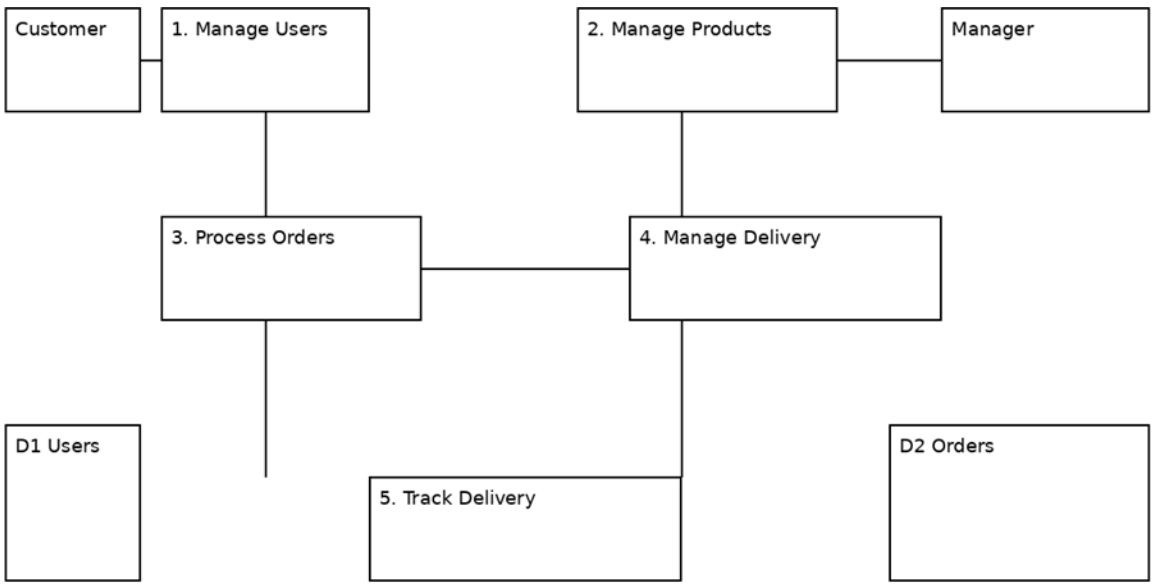
System

Grocery Delivery Management System

Data Flows

- Customer → Order Requests
- Customer → Product Searches
- Manager → Product Updates
- Manager → Delivery Assignments
- Delivery Staff → Delivery Updates
- System → Order Confirmations
- System → Delivery Status
- System → Notifications

Data Flow Diagrams (DFD)



Level 0 Processes

1. Manage Users
2. Manage Products and Inventory

3. Process Orders
4. Manage Delivery Assignments
5. Track Delivery Status

Data Stores

- User Data
- Product Inventory
- Orders
- Delivery Records
- Notifications

Object Model (Class Diagram)

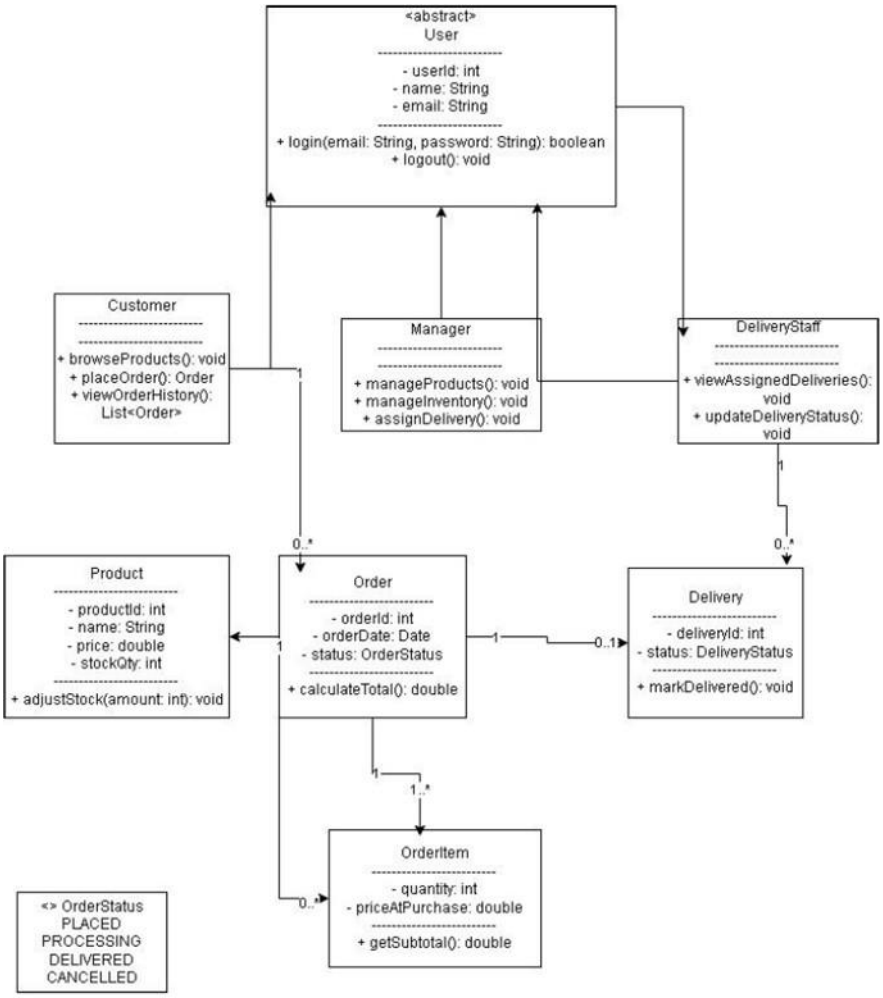


Figure: Grocery Delivery Management System Class Diagram

Classes

User (abstract)

- userId
- name
- email
- password
- login()
- logout()

Customer

- browseProducts()

- placeOrder()
- viewOrderHistory()

Manager

- manageProducts()
- manageInventory()
- assignDelivery()

DeliveryStaff

- viewAssignedDeliveries()
- updateDeliveryStatus()

Product

- productId
- name
- price
- stockQuantity
- adjustStock()

Order

- orderId
- orderDate
- status
- calculateTotal()

OrderItem

- quantity
- priceAtPurchase
- getSubtotal()

Delivery

- deliveryId
- status
- markDelivered()

Use Case Diagram (Summary)

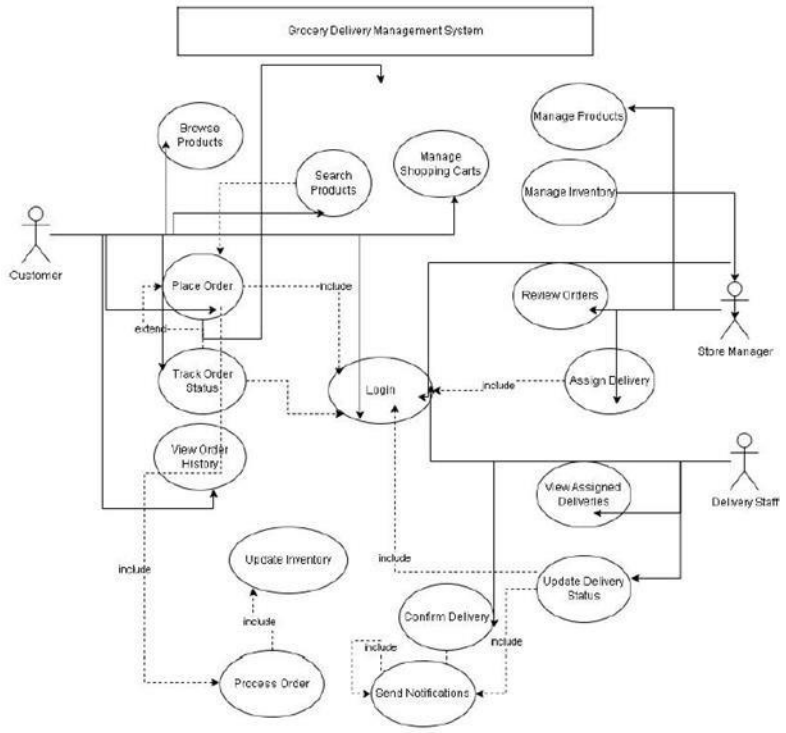


Figure: Grocery Delivery Management System Use Case Diagram

Customer

- Register / Login
- Browse Products
- Search Products
- Add to Cart
- Place Order
- Track Delivery

Manager

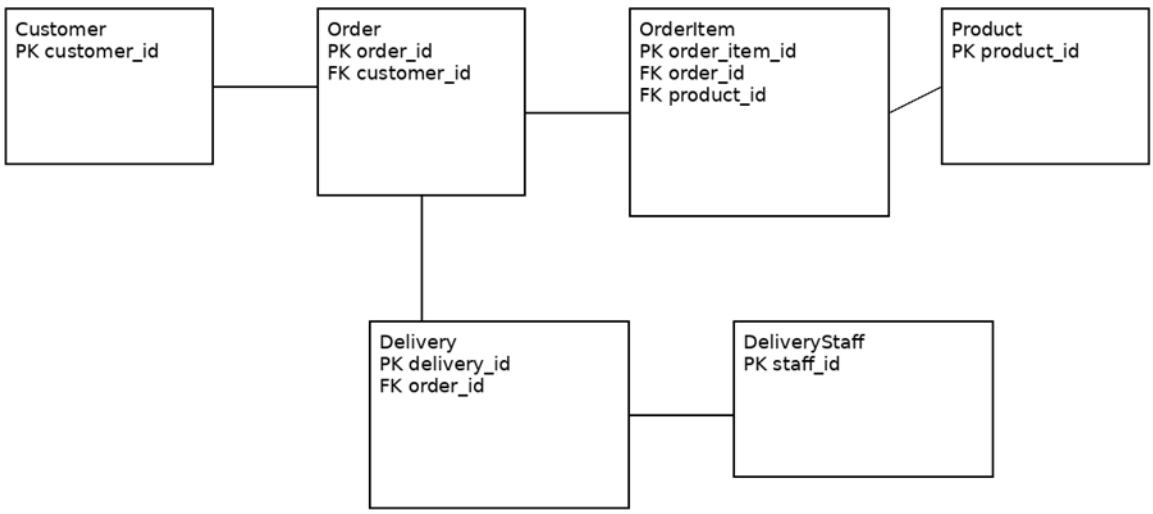
- Manage Products

- Update Inventory
- Review Orders
- Assign Delivery

Delivery Staff

- View Assigned Deliveries
- Update Delivery Status

ER Diagram (Summary)



Entities

- Customer
- Product
- Order
- OrderItem
- Delivery

- DeliveryStaff

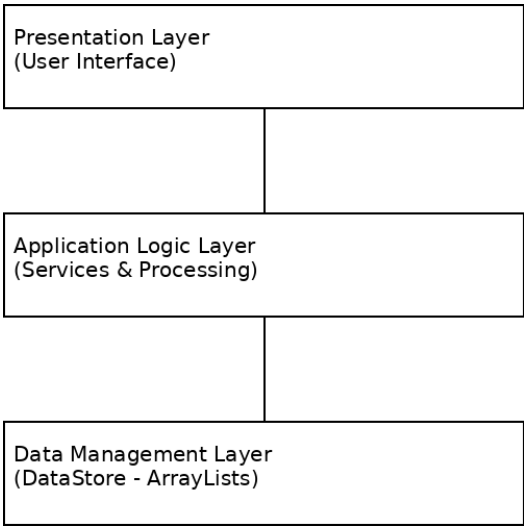
Relationships

- Customer → Order (1 to many)
- Order → OrderItem (1 to many)
- OrderItem → Product (many to 1)
- Order → Delivery (1 to 1)
- Delivery → DeliveryStaff (many to 1)

Part C: Architecture and User Interface

System Architecture

The system follows a layered architecture with three main components:



Presentation Layer
Handles all user interaction through graphical screens.
Examples:

- Login screen
- Product browsing screen
- Shopping cart
- Order tracking

Application Logic Layer
Handles system processing and business rules.
Examples:

- Order processing
- Inventory updates
- Delivery assignment
- Notification handling

Data Management Layer
The system uses **in-memory data storage**.
Data is stored using **Java ArrayLists within a DataStore class**.
Stored data includes:

- Users
- Products
- Orders
- Order items
- Deliveries

This approach makes the system easier to build and works well for a prototype. Since the data is stored in memory, it will not be saved once the application is closed.

Logical Architecture
The system is organized into three main parts:
User Interface
↓
Application Logic
↓
Data Storage (ArrayLists)

Physical Architecture
The system is designed to run on a single computer, where all parts of the application operate together. It does not require a network connection to function.

User Interface Mockups (Summary)
Login Screen

- Email field

- Password field
- Login button
- Create account option

Customer Screen

- Browse products
- Search products
- Add to cart
- Place order

Manager Screen

- Manage products
- Update inventory
- Assign deliveries

Delivery Staff Screen

- View deliveries
- Update status

Navigation Flow

Login → Dashboard → Products → Cart → Place Order → Track Delivery
The interface is designed to be simple and easy to use with minimal steps required to complete tasks.

Part D: Security and Risk Considerations

Security Features

- Role-specific access control
- Secure login authentication
- Input validation to prevent errors
- Controlled access to system data

Risks and Mitigation

One potential risk is unauthorized access to the system. This can be reduced by requiring users to log in and by limiting what each user can do based on their role.
Another risk is data loss since the system currently stores information in memory. To address this in the future, the system can be updated to use a database so data is saved permanently. There is also a risk of errors occurring during the ordering process. This can be minimized by adding checks and confirmation steps to make sure orders are correct before they are completed.

Part E: Design Patterns and Best Practices

Model-View-Controller (MVC)

Separates system into:

- Model (data)
- View (interface)
- Controller (logic)

Why used: improves organization and maintainability

Singleton Pattern

Used for DataStore class.
Why used: Makes sure only one instance manages system data

Conclusion

The Grocery Delivery Management System merges structured system design techniques with a layered architecture to support grocery ordering and delivery operations. The system combines user interaction, system processing, and data storage into a unified design that is efficient, scalable, and easy to understand.
The use of in-memory data storage simplifies development while still supporting all required system functions. The design also allows for future improvements such as database integration and web-based deployment.

References

Tilley, S., & Rosenblatt, H. (2017).

Systems Analysis and Design (11th ed.).

Cengage Learning.

Draw.io Diagram Tool

https://app.diagrams.net